17-214/514: Agentic Software Development

Containers and the Cloud

Hammad Ahmad and Claire Le Goues • Wednesday, November 11, 2026 • Lecture

Administrivia

- A5 is due Monday.
  - It is also the last chance for you to use late days. Reminder that you cannot use late days for A6.

What did we talk about last time?

- A network adds latency, lost messages, and **partial failure**. After a timeout, we can't know whether the other side got the request.
- A sync call is simple but couples the caller to the service it calls. With a queue, the caller doesn't wait, and a full queue needs backpressure.
- A transactional outbox commits a message with the data, and a relay sends it at least once.
- Retries can create duplicates, so handlers should be idempotent. At-least-once delivery plus idempotent handlers gives effectively-once processing.
- Coordination code should have tests that deliver twice, reorder, and crash partway through.
- Today: where these services run, and what makes them easy to run.

Learning goals for today

- Explain what a container image packages, and how that addresses "works on my machine"
- Review a Dockerfile or a Compose file, including one an agent generated
- Place a workload at a level of cloud abstraction, and explain what you're still responsible for
- Keep state out of disposable containers, and explain why databases don't roll back like code
- Design a service that is easy to deploy, restart, and replace

"Idk, it worked on my machine"

Why is shipping the environment harder than shipping the code?

- Your laptop may have a particular Node version, an environment variable you set once, and a system library (e.g., OpenSSL) you installed and probably forgot about.
- Production may differ in any of these, so the same code can behave differently there.
- Why not ship the environment along with the code?



ProgrammerHumor.io

Containers

The environment problem

- Deploying means running your code on machines you don't use directly. Everything it relies on (versions, paths, config, permissions) has to be set up there on purpose.
- Recall Hyrum's Law. Code often depends on parts of its environment that may not be documented.

Your laptop

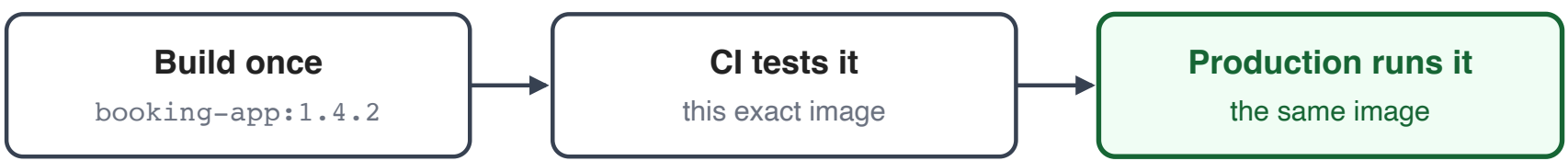
- your code
- Node 22
- PAYMENT_URL set
- libssl 3

Production

- your code (the same)
- Node 20
- PAYMENT_URL missing
- libssl 1.1

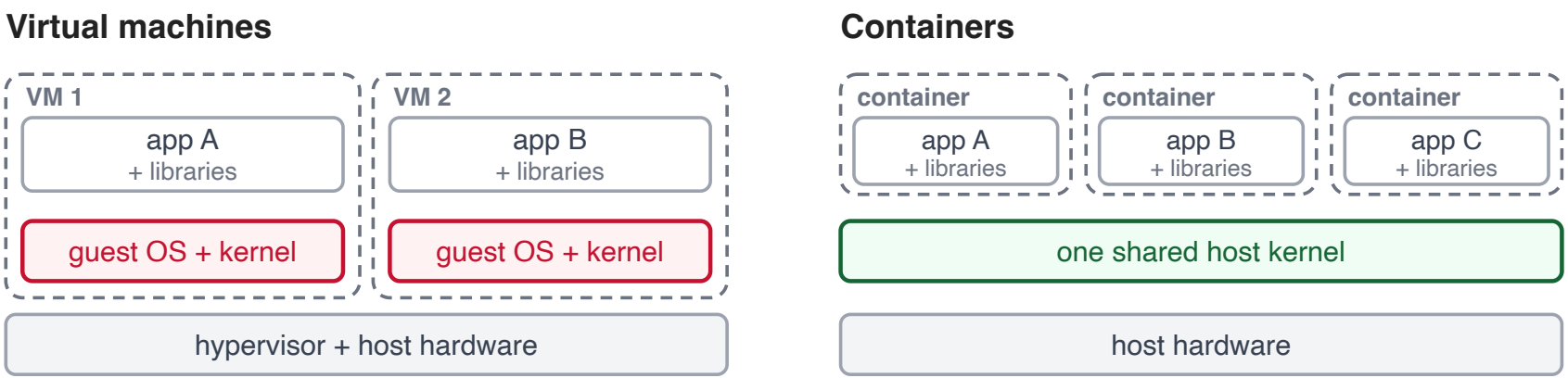
Container images

- A **container image** packages your code with its runtime, libraries, and files. A **container** is one running copy of an image.
- A container is an ordinary process with its own view of the filesystem, processes, and network. It shares the host's **kernel** (the core of the operating system, which runs processes and controls access to the hardware).
- The same image (the one that passed the tests) should run on your laptop, in CI, and in production.
- What's left to differ is config, which should be injected when the container starts (via environment variables) instead of built into the image.



Containers vs. virtual machines

- A virtual machine runs a whole guest operating system, with its own kernel, on a **hypervisor** (software that splits one physical machine into several virtual ones). A container shares the host's kernel.
- Containers usually start in seconds or less, and many more fit on one machine.
- Sharing a kernel also means lighter isolation. (Recall that a dev container and a VM were both options for sandboxing a coding agent.)



Lab 4's Dockerfile

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /build
COPY service/pom.xml ./pom.xml
COPY service/src ./src
RUN mvn -B -DskipTests package

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /build/target/lab04-service.jar ./lab04-service.jar
COPY run.sh ./run.sh
RUN chmod +x ./run.sh && useradd --system --no-create-home --shell /usr/sbin/nologin service
USER service
ENV PORT=8080
CMD ["sh", "run.sh"]
```

- One stage builds the jar, and a smaller image with only a Java runtime (JRE) runs it.
- `USER service` runs the process as a non-root user.
- Each `RUN` and `COPY` adds a **layer** to the image, cached until it or an earlier step changes.

Registries

- A **registry** stores images so other machines can pull them, like GitHub does for code (e.g., Docker Hub, or GitHub's own `ghcr.io`).
- Images are named by tag. You can deploy `booking-app:1.4.2` instead of "whatever built today".
- The tag `latest` moves whenever someone pushes a new `latest`. Pinning a version tag (or the image's **digest**, which is a hash of its contents that never changes) keeps what you deploy fixed.

```
# moves with each Node 22 release
FROM node:22
# one version (can be re-pushed)
FROM node:22.23.4-slim
# these exact bytes
FROM node:22.23.4-slim@sha256:5ec23b7c…
```

The cloud

Renting computers?

- The cloud rents you computers (e.g., Amazon EC2) on demand, plus services built on top of them.
  - For example, managed databases (Amazon RDS), object storage (Amazon S3), and queues (Amazon SQS).
- Much of what you pay for is not having to deal with what's underneath (power, failed hardware, networking, patching).
- You can pay as you go, commit to a year or more of use for a discount, or use spare capacity (which the provider can take back on short notice) at a steeper discount (**spot instances**).
  - Spot instances suit work that can be interrupted, such as batch jobs.



FSFE, Markus Meier, CC BY-SA 4.0

Cloud costs

- Many cloud resources (machines, disks, databases) are billed while they exist, whether or not they're doing useful work. A forgotten test machine is still billed for every second it runs.
- Instance sizes, storage, data transfer, and "did anyone tear that down?" should come up in design reviews, before they show up on the invoice.
- It's easier to build the teardown habit while the bills are small or nonexistent.
  - That's why Lab 4 and A2 asked you to tear down every time.



ProgrammerHumor.io

Levels of abstraction

Virtual machines

You get a machine. You're responsible for the OS, runtime, deploys, and patches.

e.g., Amazon EC2

Managed containers

You hand over an image. The platform places it and restarts it.

e.g., Amazon ECS on AWS Fargate

Serverless functions

You hand over a function. There's no process to manage, and little to configure.

e.g., AWS Lambda

← more to manage, more control

more cost per unit of work, more tied to one provider →

Which level for the booking API? It depends (on how much you want to operate, how much control you need, and how the cost grows with traffic).

Shared responsibility

- The provider handles the hardware, power, physical security, and (depending on the level) some of the software above them.
- At every level, you're still responsible for your code, data, access keys, config, your dependencies' vulnerabilities, etc.
- Each provider writes down where its part ends. Problems often show up where each side assumed the other was responsible…

The 2024 Snowflake breaches

- In 2024, attackers logged into many companies' Snowflake accounts with stolen passwords.
- Mandiant found no sign that Snowflake's own systems were breached. The passwords came from malware on customers' or contractors' machines.
- The accounts had no multi-factor authentication, and some passwords hadn't been changed in years. Under Snowflake's shared-responsibility model, both were the customers' job.

> Mandiant's investigation has not found any evidence to suggest that unauthorized access to Snowflake customer accounts stemmed from a breach of Snowflake's enterprise environment. Instead, every incident Mandiant responded to associated with this campaign was traced back to compromised customer credentials.

> The impacted accounts were not configured with multi-factor authentication enabled, meaning successful authentication only required a valid username and password.

Mandiant, June 10, 2024

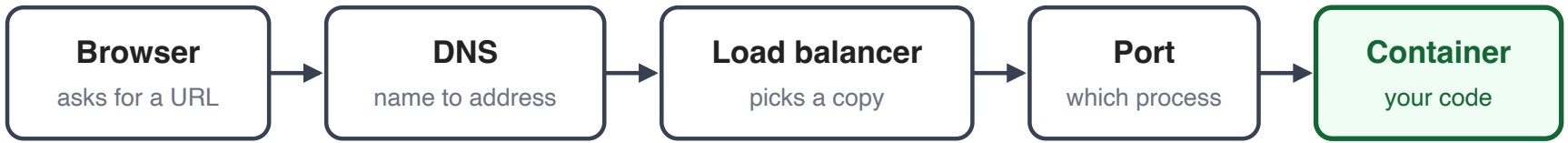
Regions and zones

- Your service runs in a physical place. A **region** is a group of data centers in one area, divided into **availability zones** that can fail independently.
  - For example, the AWS region `us-east-1` (Northern Virginia) has six zones, `us-east-1a` through `us-east-1f`.
- Latency grows with distance to your users.
- Some data may not legally leave its country.
- Replicas in several zones can keep a service up when one zone fails.

> AWS has the concept of a Region, which is a physical location around the world where we cluster data centers. We call each group of logical data centers an Availability Zone. Each AWS Region consists of a minimum of three, isolated, and physically separate AZs within a geographic area. …

AWS, Regions and Availability Zones

The path of a request



- Each hop has its own configuration. Any hop can be the broken one.
- When the site is down, check the hops in order.

```
# on your machine:
dig booking.example.com       # DNS: which address?
curl http://<address>:8080/   # load balancer, port: does it answer?
# then, on the server (e.g., over SSH):
docker ps                     # container: running? ports mapped?
docker logs booking-api       # container: what did it log?
```

Lab 4's deployment template

- It rented one small virtual machine.
- It opened the service port to the internet with a **security group** (AWS's firewall rules for which ports a machine accepts traffic on, and from where).
- The URL was the machine's public DNS name, which AWS assigns.

```yaml
# template.yaml (excerpt)
ServiceSecurityGroup:
  Type: AWS::EC2::SecurityGroup
  ...
        FromPort: !Ref ServicePort
        ToPort: !Ref ServicePort
        CidrIp: 0.0.0.0/0
  ...
ServiceInstance:
  Type: AWS::EC2::Instance
  Properties:
    InstanceType: t3.micro
    ...
Outputs:
  ServiceUrl:
    ...
    Value: !Sub "http://${ServiceInstance.PublicDnsName}:..."
```

Lab 4's boot script

- On boot, the machine installed Docker and started the course's image.
- `--restart unless-stopped` starts the container again if it crashes or the machine reboots, unless someone stopped it on purpose.
- `-p` forwards the service port from the machine to the container.
  - A published port is reachable from outside the machine, by anyone the security group lets in. (In a Docker Compose file, `ports:` does the same.)

```
# template.yaml, UserData (excerpt)
dnf install -y docker
systemctl enable --now docker
shutdown -h +240
docker run -d \
  --name lab04-service \
  --restart unless-stopped \
  -p ${ServicePort}:${ServicePort} \
  -e PORT="$EFFECTIVE_PORT" \
  ${ImageUrl}
```

- `-e PORT=...` sets an environment variable that tells the **service** which port to listen on inside the container.

Lab 4's broken deploy

Recall scenario 2 from Lab 4. These are the lab's outputs:

```
$ curl http://ec2-54-160-135-197.compute-1.amazonaws.com:8080/api/health
curl: (7) Failed to connect to ... port 8080 after 67 ms: Couldn't connect to server
sh-5.2$ sudo docker ps
...   PORTS                                       NAMES
...   0.0.0.0:8080->8080/tcp, :::8080->8080/tcp   lab04-service
sh-5.2$ sudo docker logs lab04-service
lab04-service listening on 9090
```

- The name resolved (a DNS failure says "Could not resolve host"), but the connection to port 8080 failed.
- On the machine, `docker ps` shows 8080 forwarded to 8080, but the log says the service listens on 9090. The broken hop is the port.
- The fix went into the template's parameters. The stack was recreated instead of being patched in place. (We'll see why next.)

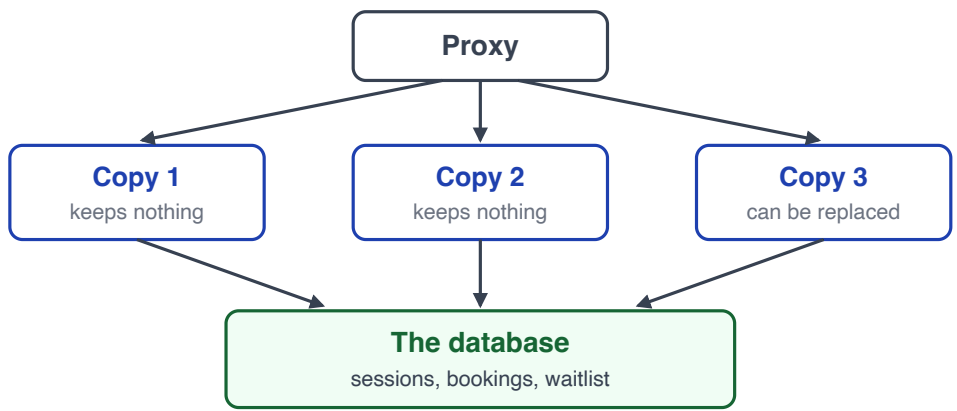
Infrastructure as code

- The environment itself (machines, networks, ports) is declared in files in the repo and should be reviewed like the rest of the code.
  - For example, Lab 4's `template.yaml` (a CloudFormation template) declares the machine, the security group, and the URL.
- That makes it repeatable and diffable, like the Dockerfile is for the image.
- It can drift the same way docs do (e.g., when someone changes the live system by hand). Changes should go through the files, with the console treated as read-only.

State

Reminder: state needs a home

- A stateless service keeps nothing between requests, so any copy can serve any request.
- State in one copy's memory makes that copy special. It can't be replaced or duplicated without losing that state.
- The state should go into systems built for it (databases, object stores, caches). Scaling out often moves the bottleneck there.



Volumes

- A container's own filesystem goes away when the container is removed. That's fine for a stateless app.
- For a database container, a replacement container may start without the old data.
- A **volume** is storage mounted from outside the container. The data outlives every container that uses it.
- For example, Sentry's self-hosted Compose file gives Postgres a named volume and a health check.

```
  postgres:
    ...
    image: "postgres:18.6-trixie"
    healthcheck:
      ...
      test: ["CMD-SHELL",
             "pg_isready -U ${POSTGRES_USER:-postgres}"]
    ...
    volumes:
      - "sentry-postgres:/var/lib/postgresql"
    ...
volumes:
  ...
  sentry-postgres:
    external: true
```

Sentry's self-hosted docker-compose.yml (github.com/getsentry, trimmed)

Databases don’t roll back like code

- Rolling back stateless code is cheap. Deploy the previous image, and you’re back.
- The data doesn’t roll back that way.
  - Recall that a breaking schema change can be split into three steps (expand, migrate, contract). Expand steps are usually easy to roll back. Contract steps often aren’t.

Let's practice: an agent reviews a Compose file

Canvas → Quizzes →
Lecture 19 In-Class Quiz

```yaml
services:
  api:
    image: booking-app:1.4.2
    env_file: .env
    ports: ["8080:8080"]
    depends_on: [db]
    restart: unless-stopped
  db:
    image: postgres:16.4
    environment:
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    ports: ["5432:5432"]
```

The agent's findings:

1. The database password comes from the environment (`${DB_PASSWORD}`), so it isn't stored in this file.
2. `db` has no volume, so its data is lost when the container is removed.
3. Port 5432 is published, so the database is reachable from outside the machine.
4. `db` has no restart policy.

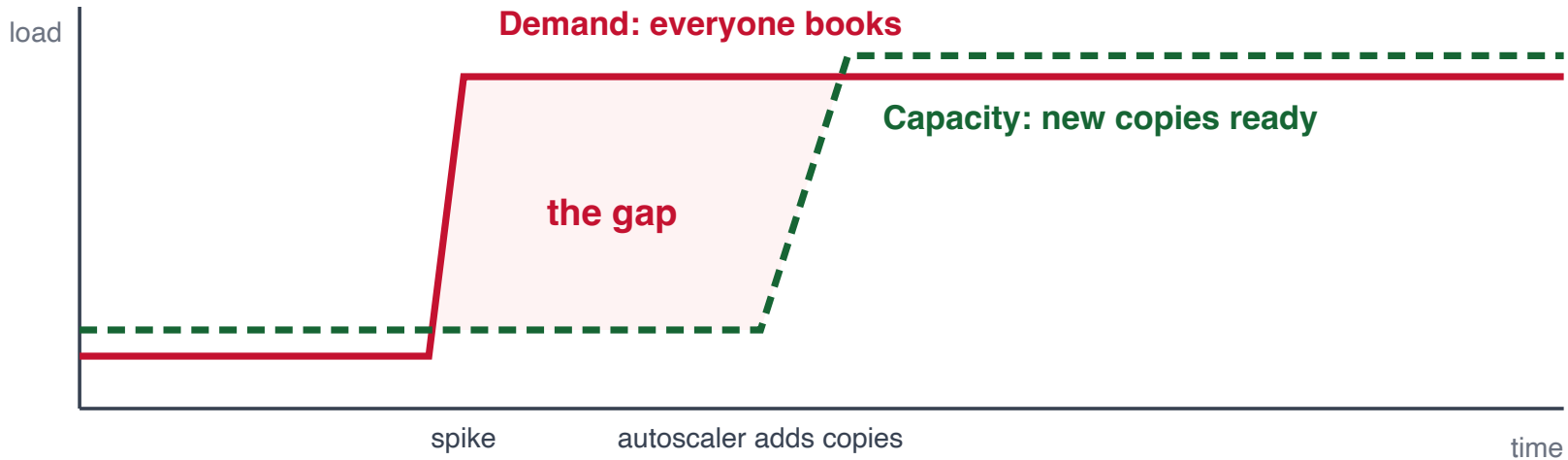
With a neighbor: which finding would you fix first, and why? Is any finding wrong or incomplete?

Scaling

Autoscaling lags

- Autoscaling reacts to load after it arrives. Noticing the load, starting new copies, and warming them up all take time.
- The first requests to a new copy are often slower. This is called a **cold start**.
  - Serverless functions hit this too. After a quiet period, the platform may have no copies running, so the next request waits for one to start.
- Traffic can spike faster than new copies start.

What if everyone books at once?



What happens in the gap is a design decision. Requests can time out, but retries would add more load. The system can also queue the work, or reject some of it with a clear error.

Orchestration

- Many containers need placing, restarting, and replacing when a machine dies.
- An orchestrator (e.g., Kubernetes) takes a **desired state**, such as "six copies of this image". It keeps checking what's running and starts or stops containers until the two match.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: { name: booking-api }
spec:
  replicas: 6
  selector: { matchLabels: { app: booking-api } }
  template:
    metadata: { labels: { app: booking-api } }
    spec:
      containers:
        - { name: api, image: "booking-app:1.4.2" }
```

- Compose does a small version of this. You declare the stack, and it starts it.
  - For example, A5's `docker compose up --scale api=4` starts four copies. Unlike Kubernetes, Compose doesn't keep checking. (A copy that dies comes back only through its restart policy.)

Design for deployability

Something fails all the time

- At enough scale, some disk, machine, or zone is usually failing somewhere.
- At that scale, systems should recover on their own, with replicas, health checks, and automatic restarts.
- The platform does that best when your service reports its health, and starts and stops quickly.

> Several AWS services experienced impact to the control planes that are used for creating and managing AWS resources. These control planes use services hosted in the internal network. For example, while running EC2 instances were unaffected by this event, the EC2 APIs that customers use to launch new instances or to describe their current instances experienced increased error rates and latencies starting at 7:33 AM PST. …

AWS, US-EAST-1 event summary, December 2021

- Some failures span a whole region. In AWS's 2021 outage, running machines kept running, but new ones often couldn't be launched because the AWS systems that launch them were impaired.
  - Spreading across zones didn't help. Recovery that needs new machines (scaling up, replacing a failed copy) can stall at exactly the wrong time.

Operability is a design property

- Whether a service is easy to operate is mostly decided in its code, long before an incident.
  - Can it start fast?
  - Stop without losing work?
  - Report whether it's healthy?
- A service that keeps state in memory, takes minutes to start, and needs a careful shutdown makes orchestrators and autoscalers harder to use.
- Like testability and changeability, operability is easier to design in than to add later.

Twelve-factor habits

The Twelve-Factor App (Adam Wiggins at Heroku, 2011) is a checklist for services that run well on a cloud platform. Three of its habits:

Config from the environment

As before, the image should be the same everywhere, and environment variables carry what differs. No `config_prod_final2.js`.

Logs to stdout

Your code prints, and the platform collects the output.

Disposability

Fast startup and graceful shutdown, so any copy can be stopped at any time.

Health checks

- A **health check** is a request or command the platform runs to ask a container whether it's OK.
  - For example, your Better Slack's `GET /api/health`, which A5 checks through the proxy.
- Kubernetes asks two separate questions. A failed **liveness** check restarts the container. A failed **readiness** check stops traffic to it, without a restart.

```yaml
services:
  api:
    image: booking-api:1.4.2
    ports: ["3000:3000"]
    healthcheck:
      test: ["CMD", "node", "healthcheck.js"]
      interval: 5s
      retries: 3
```

```
$ docker compose ps
NAME       STATUS
hc-api-1   Up 8 seconds (healthy)
```

- Plain Compose can mark a container unhealthy, and other services can wait until it's healthy.
  - It doesn't restart an unhealthy container.

Graceful shutdown

- To stop a container, Docker sends `SIGTERM` (a signal asking the process to stop), waits a grace period, then sends `SIGKILL` (which ends it immediately).
  - During A5 grading, we may stop one of your `api` replicas mid-test.
- Without a handler for `SIGTERM`, the booking API may ignore the signal and be killed after the grace period.
  - A request in progress at that moment would be cut off.
- With the handler below, the health check fails, the server stops taking new connections, open requests finish, and the process exits.

```
process.on("SIGTERM", () => {
  ready = false;                          // health checks now fail
  server.close(() => process.exit(0));  // exits once open requests finish
});
```

Secrets

- Passwords and API keys should be injected when the container starts, from a secret store (e.g., AWS Secrets Manager). They don't belong in the image or the repo.
- An image is made to be shared (pushed, pulled, cached, inspected). A secret built into a layer goes everywhere the image goes.
- Rotating a key is a good test. If changing it means rebuilding the image, the key is in the wrong place.

Built into the image (in every copy of it)

```
ENV STRIPE_KEY=sk_live_4f...
```

Injected at start (rotating doesn't need a rebuild)

```
docker run --env-file prod.env \
  booking-api:1.4.2
```

Checking an image for secrets

Suppose the booking API's Dockerfile sets `ENV STRIPE_KEY=...`. The image's history then shows:

```
$ docker history --format '{{.CreatedBy}}' booking-api:1.4.3
CMD ["node" "server.js"]
USER node
ENV STRIPE_KEY=sk_live_4f...
COPY . . # buildkit
RUN /bin/sh -c npm ci --omit=dev # buildkit
...
```

- Anyone who can pull the image can read the key. `docker history` shows each build step, and `docker image inspect` shows the environment.
- The fix is a rebuild without that line, plus a new key, since copies of the old image may still be out there.

Reviewing generated deployment files

A generated Dockerfile

Asked for "a Dockerfile for this Node service", an agent might write:

```
FROM node:22
WORKDIR /app
COPY . .
RUN npm install
EXPOSE 3000
CMD ["npm", "start"]
```

- It builds and runs, but it has problems that a functional test wouldn't catch.
- `node:22` is the full image (much larger than `node:22-slim`). Also, the tag moves with each Node 22 release.
- `COPY . .` copies all the code, so any code change invalidates the cache from there on, and `npm install` reruns every time. Without a `.dockerignore`, it also copies `.env` and `.git`.
- `npm install` adds dev dependencies. With no `USER` line, the process runs as root.

The Dockerfile after fighting with your agent

```
FROM node:22.23.4-slim
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
USER node
CMD ["node", "server.js"]
```

- A slim base image that is pinned to one version.
- Dependencies before code, so `npm ci` reruns only when the package files change. `--omit=dev` leaves out dev dependencies.
- A `.dockerignore` file (not shown) listing `node_modules`, `.env`, and `.git` keeps those out of `COPY . .`.
- `USER node` runs the process as `node` (a non-root user the official Node images include).

Measuring both images

Both images built, then rebuilt after a one-line code change:

```
$ docker image ls booking-api
IMAGE                    ...   DISK USAGE
booking-api:fixed        ...        251MB
booking-api:generated    ...       1.18GB
$ docker run --rm booking-api:generated whoami
root
$ docker run --rm booking-api:fixed whoami
node
```

Rebuilding the generated image

```
#8 [4/4] RUN npm install
#8 DONE 0.7s
```

Rebuilding the reviewed image

```
#7 [4/5] RUN npm ci --omit=dev
#7 CACHED
```

- The full `node:22` base accounts for most of the size difference.
- After the change, the reviewed build reused its installed dependencies, but the generated build ran `npm install` again (usually the slow step).
- The generated image runs as root.

Golden paths

- A **golden path** is a starting point with safe defaults (a base image, a Dockerfile template, an infrastructure module). Instead of reviewing every generated file from scratch, you can point the agent at one.
- The diff is then smaller and follows a template you already know.
- The agent's instruction file can point at the templates.

```
## Containers
- Start from docker/Dockerfile.template (pinned slim base, non-root user).
- Secrets come from env_file at start. Never put one in a Dockerfile or compose file.
```

Let's practice: three copies and a deploy

```python
sessions = {}  # session id -> user

@app.post("/login")
def login():
    sid = str(uuid.uuid4())
    sessions[sid] = request.json["user"]
    resp = make_response("", 204)
    resp.set_cookie("sid", sid)
    return resp
...
@app.post("/rooms/<id>/photo")
def upload_photo(id):
    with open(f"uploads/{id}.jpg", "wb") as f:
        f.write(request.data)
    return "", 201
...
@app.get("/health")
def health():
    return "", 200
```

> **The agent's summary:** Sessions are stored in memory. Uploaded room photos are saved to `./uploads`. `GET /health` returns 200 while the server is running. Ready to deploy.

The platform runs three copies behind a load balancer and replaces one copy during a deploy.

1. A user logs in. What do their next few requests see?
2. A user uploads a room photo, and later the copy that handled the upload is replaced. Where's the photo?
3. What does the summary leave out that matters during the deploy?

Problems with the agent's service

With three copies behind a load balancer:

1. The session lived in one copy's memory. Only about one request in three reached that copy, so the rest came back 401. Sessions should live in a database or a cache.
2. The photo was only on the copy that handled the upload. After that copy was replaced, every copy returned 404. A volume or object storage (e.g., Amazon S3) keeps it.
3. The summary doesn't mention shutdown. There's no `SIGTERM` handler, so a request in progress when its copy was stopped was cut off.
4. Should `/health` check the database? For liveness, no. Restarting the app doesn't fix a database outage, and every copy would restart. For readiness, yes. A copy that can't reach the database shouldn't get traffic.

Summary

- A container image packages the environment with the code, so the same image should run from laptop to production. The Dockerfile makes that environment reviewable code.
- The cloud rents machines and services on top of them. Each level (VMs, managed containers, serverless) leaves you responsible for a different part.
- A request goes through DNS, a load balancer, a port, and a container. Any hop can be the broken one.
- Containers can be replaced at any time. State should live in volumes and databases, which don't roll back like code.
- Something fails all the time, so services should be easy to deploy, restart, and replace (config and secrets from the environment, health checks, graceful shutdown).
- Generated Dockerfiles and Compose files should be reviewed (base images, pins, users, ports, secrets, volumes). Golden paths make that review smaller.

What's next

W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14
A1 out		A1 due		A2 due · M1		A3 due		A4 due	M2 · A5 chkpt		A5 due	A6 chkpt	A6 due

- **Friday, Lab 11:** Events, replay, and refunds.
- **Monday, Nov 16:** A5 due at 11:59 pm. Also the DevOps lecture (the pipeline that takes a commit to production).
- **Next week:** A6 release. You'll be working with deployment and monitoring.